📄 SHL AI Assessment Recommendation System

Approach & Optimization Report

---

1. Problem Understanding

Hiring managers struggle to find relevant SHL assessments using keyword search. The goal of this project was to build an intelligent recommendation system that:

- Accepts natural language job descriptions
- Retrieves the most relevant "Individual Test Solutions"
- Returns structured recommendations via API
- Achieves measurable performance using Recall@K

The system must:

- Crawl and store SHL catalogue data
- Use retrieval/LLM-based techniques
- Provide evaluation metrics
- Maintain recommendation balance across domains

- **2. System Architecture**
- The solution follows a modular pipeline:

Data Crawling → Cleaning → Embedding Generation → Semantic Search → Balanced Recommendation → API → Frontend

2.1 Data Ingestion

- Crawled SHL product catalogue
- Ignored "Pre-packaged Job Solutions"
- Collected 518 assessments
- Cleaned and structured 415 usable Individual Test Solutions
- Stored attributes:
    - name
    - URL
    - description
    - test_type
    - duration
    - adaptive_support
    - remote_support

This ensures reproducibility and independence from live scraping.

---

3. Retrieval & Recommendation Strategy

3.1 Embedding-Based Semantic Search

Instead of keyword matching, the system uses:

- Sentence Transformers (all-MiniLM-L6-v2)
- Converts both queries and assessment descriptions into vector embeddings
- Computes cosine similarity
- Retrieves top-K relevant assessments

This enables:

- Context-aware matching
- Understanding of long job descriptions
- Improved relevance vs simple keyword search

---

4. Recommendation Balance Logic

To satisfy the requirement of balanced results across multiple domains:

Example:

“Need a Java developer good at collaborating”

The system:

- Detects both technical and behavioural signals
- Ensures mixture of:
    - Knowledge & Skills (K)
    - Personality & Behaviour (P)
    - Ability & Aptitude (A)

This is achieved by:

- Including test type as part of metadata
- Re-ranking results to prevent dominance of single category

This improves recommendation diversity and realism.

---

5. Evaluation Strategy

Evaluation was performed using the provided labelled train set.

Metric Used:

Mean Recall@K

$$Recall@K = \frac{\text{Relevant in Top K}}{\text{Total Relevant}}$$

$$MeanRecall@K = \frac{1}{N} \sum Recall@K_i$$

---

6. Performance Optimization Journey

Initial Version

- Basic cosine similarity
- Mean Recall@10 = 0.0200

Issues:

- Weak semantic matching
- No balancing logic

Improved Version

- Better preprocessing
- Cleaned descriptions
- Removed noisy text
- Tuned top_k parameter

Result:

- Mean Recall@10 = 0.1467

Further Improvement

- Increased K to 20
- Re-ranking for diversity
- Balanced test types

Result:

- Mean Recall@20 = 0.2189

This demonstrates iterative optimization and measurable improvement.

---

7. API Implementation

Two endpoints implemented:

GET /health

Returns:

```
{ "status": "ok" }
```

POST /recommend

Input:

```
{
  "query": "string",
  "top_k": 10
}
```

Output:

```
{
  "recommended_assessments": [
    {
      "url": "...",
      "name": "...",
      "adaptive_support": "Yes/No",
      "description": "...",
      "duration": 45,
      "remote_support": "Yes/No",
      "test_type": ["Knowledge & Skills"]
    }
  ]
}
```

All responses follow proper HTTP status codes and JSON format.

---

8. Frontend

A Stream lit-based UI was built to:

- Accept job descriptions
- Display top recommendations

- Allow adjustable number of results

This enables interactive validation.

---

9. Reproducibility & Code Structure

The project is modular:

- src/
- ├── recommender.py
- ├── evaluate.py
- ├── generate_test_predictions.py
- app.py
- frontend.py
- data/
- screenshots/
- Separation of concerns
- Clean pipeline
- Evaluation scripts
- Submission CSV generation script

---

10. Final Deliverables

- Functional API
- Public GitHub repository
- Frontend application
- Submission CSV file (test_predictions.csv)
- Evaluation proof
- Screenshots for verification